

FUNCTIONS

→ Functions is a reusable block of code, which takes input, process it and give output

• `def function_name (input arguments):`

<body code>

↓
optional

return
Optional

→ If the fn is not returning anything, then its return type is None.

⇒ `def pythonfun():`
 `print("Hello")`
 `l = [10, 20, 30, 40]`
 `return l`

`a = pythonfun()`
`print(a)`

O/p → Hello
 [10, 20, 30, 40]

⇒ `def fun(x):`

`a = x*2`
 `return a`

• `l = fun(2)`
 `print(l)`

O/p → 4

• `l = fun("hello")`

`print(l)`

O/p → hellohello

• `l = fun([1, 2, 3])`

`print(l)`

O/p → [1, 2, 3, 1, 2, 3]

* Positional Arguments

Order is very imp.

`def employee_info (name, age, design):`

`print("Name:", name)`

`print("Age:", age)`

`print("Designation:", design)`

`a = employee_info("Ravi", 21, "Student")`

`print(a)`

Name: Ravi

Age: 21

Designation: Student

None

→ Positional Arguments

→ Keyword Arguments

→ Default Parameters

→ Positional only & keyword

(Single *)

FUNCTIONS

⇒ To pass any no. of positional arguments (Stores in tuple form)

```
def fun(*args):  
    print(args)  
    print(type(args))  
  
fun(1, 10, 7.5, "hello", [1, 2, 3])
```

O/P → (1, 10, 7.5, "hello", [1, 2, 3])
<class, tuple>

```
# print(args[3])
```

↳ ("hello")

★ Keywords arguments [Order does not matter]

```
def employee_info(name, age, desig.):  
    print("Name:", name)  
    print("Age:", age)  
    print("Designation:", desig)  
  
employee_info(age=21, desig="Student", name="Anmol")
```

O/P
Name: Anmol
Age: 21
Designation: Student
(Double **)

⇒ To pass any no. of keyword arguments (Stores in dict. form)

```
def fun(**kwargs):  
    print(kwargs)  
    print(type(kwargs))  
  
fun(name="Anmol", age=21, salary=100000)
```

O/P → {"name": "Anmol", "age": 21, "salary": 100000}
<class, dict>

```
def fun(*s, **k):  
    print(s) # tuple return  
    print(k) # dict return
```

```
fun(10, 7, 9, 11, 4.5, a=100, b="Hello", c=91.2)
```

O/P

↳ (10, 7, 9, 11, 4.5)
{"a": 100, "b": "Hello", "c": 91.2}

★ Default Parameters

```
def country_name (country = "India"):
    print (f"my my country name is is {country}")
country_name()
```

o/p → my country name is India.

```
country_name ("Russia")
o/p → my country name is Russia.
```

∴ Default arguments will always come after non-default arguments.

```
def fun(a, b=1):
    print(a, b)
```

```
→ fun(1) → 1, 1
→ fun(1, 2) → 1, 2
```

★ Positional only argument

```
def fun(a, /):
    print(a)
fun(3) → 3
fun(a=3) → Error
      ↓
    Keyword
```

★ Keyword only argument

```
def fun(*, a):
    print(a)
fun(3) → Error
fun(a=3) → 3
```

★ Combining positional only arguments & keyword only arguments

```
def fun(a, b, /, c, *, d, e):
    # positional arguments: a, b
    # both positional & keyword arguments: c
    # keyword arguments: d, e
```

Que

```
def (a, b, /, *, c, d):
    print(a, b, c, d)
```

- × (i) fun (10, 20, 30, 40) # all pos. ×
- × (ii) fun (10, 20, 30, d=40) # 3 pos & 1 key ×
- ✓ (iii) fun (10, 20, c=30, d=40) # 2 pos & 2 key ✓
- × (iv) fun (a=10, b=20, c=30, d=40) # all key ×

- `def fun(a, /, b, *, c):`

- `print(a, b, c)`

- `fun(10, 30, c=20)`

- `fun(10, b=30, c=20)`

★ Docstring (Just placed exactly after/below function)

⇒ It shows that what function do. (Written in `"""..."""` exactly after for declaration)

⇒ Only shows using `help(function-name)` called...

- `def fun(a, b):`

- `"""The function is doing add, sub, mul of two numbers"""`

- `print(a+b)`

- `print(a-b)`

- `print(a*b)`

∴ `help(fun)`

★ Local Variables

⇒ Local variables are those variables which are defined inside any function and they belong to that function.

⇒ They cannot be accessed outside the function.

- `def fun():`

- `a = 100`

- `print(a)`

- `fun()`

- `print(a)`

→ Local Variable

O/p ⇒ 100
Error

★ Global Variables

⇒ Global variables are those variables which are defined outside any function and can be accessed anywhere in the program (i.e. inside & outside the function)

- `a = 100`

- `def fun():`

- `print(a)`

- `fun()`

- `print(a)`

→ Global Variable

O/p ⇒ 100
100

- `a = 1000`

- `def fun():`

- `a = 10`

- `print(a)`

- `fun()`

- `print(a)`

O/p ⇒ 10
1000

Note:

• `d=100`
`def fun():`
`d = d + 500 → Error`
`print(d)`

`fun()`
`print(d)`

By default functions can access the global variable but cannot modify the global variable

But := "global" keyword is used if we want to modify the global value inside function

• `d=100` 600
`def fun():`
`global d`
`d = d + 500`
`print(d)`

`fun()`
`print(d)`

d = 600
 O/p ⇒ 600
 600

⇒ If there is no global variable declared then it will create one. if it is declared in function.

Q17 `a=50`
`def func():`
`a=100`
`func()`
`print(a)`
 ⇒ 50

Local Variable

Q27 `a=100`
`def func():`
`global a`
`a=1000`
`func()`
`print(a)`
 ⇒ 1000

Q37 `def func():`
`global a`
`a=1000`
`func()`
`print(a)`
 ⇒ 1000

★ Nested function (Tough Ques.) 99% Not Asked / 1% Asked

⇒ Nested function are those functions which defined inside another functions. The outer function can call the inner function and inner function can access the variables and arguments of outer functions.

• `def outer():`
`a=10`
`b=20`
`def inner():`
`print(a,b)`
`inner()`
`outer()`

⇒ 10 20

• `def outer():`
`x=5`
`def inner():`
`print(x)`
`return inner`

`a=outer()`
`a()` inner

O/p ⇒ 5

closure property : Inner fn remembers the prop. of outer fn even outer fn is completed

Inner fn remembers the property of outer fn even
where the execution of outer fn is completed. is known
as closure property.

```
def multiplier(factor):  
    def multiply-by(x):  
        return x * factor  
    return multiply-by
```

```
a = multiplier(2)  
print(a(6))  
b = multiplier(3)  
print(b(7))
```

=> 12

=> 21

O/p => 12
21

a(x)

b(x)

return
 $x * 2$

return
 $x * 3$